

MARAN SUITE SYSTEM — Especificaciones de Desarrollo: Módulo Hospitalidad

Fecha: Marzo 2026

Módulo: Hospitalidad — CRM de preferencias y experiencia del huésped

Prioridad: Alta — Feature diferenciadora de servicio personalizado

Contexto general

El módulo de Hospitalidad es el CRM interno del hotel. Permite registrar gustos, preferencias, fechas especiales y notas de cada huésped para que cualquier área del hotel pueda brindar un servicio personalizado en todo momento. Se integra con el sistema de notificaciones (ya especificado) para alertar automáticamente al hacer check-in si el huésped tiene preferencias registradas.

A diferencia de las reseñas (que son feedback del huésped hacia el hotel), la hospitalidad es conocimiento interno del staff sobre el huésped.

TAREA 1 — Schema de datos

Área: Backend | `shared/schema.ts`

Nueva tabla `guestPreferences`

Preferencias permanentes del huésped — se acumulan visita a visita y persisten en su perfil.

```
export const guestPreferences = pgTable("guest_preferences", {
  id: serial("id").primaryKey(),
  guestId: integer("guest_id").notNull().references(() => guests.id),

  // Categoría de la preferencia
  category: text("category").notNull(),
  // Categorías: "habitacion", "alimentacion", "amenities", "servicio",
  //           "fecha_especial", "motivo_viaje", "nota_interna", "otro"
```

```

// Subcategoría opcional para organizar mejor
subcategory: text("subcategory"),
// Ejemplos por categoría:
// habitacion: "piso", "vista", "temperatura", "cama", "silencio"
// alimentacion: "alergia", "dieta", "bebida_favorita", "desayuno"
// amenities: "almohada", "colchon", "aromaterapia", "minibar"
// servicio: "horario_limpieza", "no_molestar", "preferencia_idioma"
// fecha_especial: "cumpleanos", "aniversario", "otro"

title: text("title").notNull(),
// Título corto (ej: "Alérgico al maní", "Prefiere piso alto", "Cumpleaños 15 d

description: text("description"),
// Detalle adicional (ej: "Alergia severa, revisar todos los platos del restaur

isActive: boolean("is_active").notNull().default(true),
// Permite desactivar preferencias que ya no aplican sin borrarlas

priority: text("priority").notNull().default("normal"),
// Valores: "low", "normal", "high", "critical"
// critical = alergias severas, necesidades médicas, VIP especial

visibleTo: text("visible_to").array().notNull().default(["all"]),
// Áreas que deben ver esta preferencia
// Valores posibles: "reception", "housekeeping", "restaurant", "spa", "all"

recordedBy: text("recorded_by"),
// Nombre del staff que registró la preferencia

sourceStay: integer("source_stay").references(() => reservations.id),
// Reserva durante la cual se detectó esta preferencia (opcional)

createdAt: timestamp("created_at").defaultNow(),
updatedAt: timestamp("updated_at").defaultNow(),
});

```

Nueva tabla `stayNotes`

Notas específicas de una estadía puntual — no persisten en el perfil del huésped pero quedan en el historial de la reserva.

```

export const stayNotes = pgTable("stay_notes", {
  id: serial("id").primaryKey(),
  reservationId: integer("reservation_id").notNull().references(() => reservation
  guestId: integer("guest_id").references(() => guests.id),

```

```

category: text("category").notNull(),
// Mismas categorías que guestPreferences

title: text("title").notNull(),
description: text("description"),

priority: text("priority").notNull().default("normal"),
visibleTo: text("visible_to").array().notNull().default(["all"]),

isResolved: boolean("is_resolved").default(false),
// Para notas que requieren acción (ej: "Pedir cuna para bebé") – marcar como r

resolvedAt: timestamp("resolved_at"),
resolvedBy: text("resolved_by"),

recordedBy: text("recorded_by"),
createdAt: timestamp("created_at").defaultNow(),
});

```

Nueva tabla `hospitalityAlerts`

Alertas generadas automáticamente al hacer check-in basadas en preferencias del huésped.

```

export const hospitalityAlerts = pgTable("hospitality_alerts", {
  id: serial("id").primaryKey(),
  reservationId: integer("reservation_id").notNull().references(() => reservation),
  guestId: integer("guest_id").notNull().references(() => guests.id),
  preferenceId: integer("preference_id").references(() => guestPreferences.id),

  alertMessage: text("alert_message").notNull(),
  targetArea: text("target_area").notNull(),
  priority: text("priority").notNull().default("normal"),

  isAcknowledged: boolean("is_acknowledged").default(false),
  acknowledgedAt: timestamp("acknowledged_at"),
  acknowledgedBy: text("acknowledged_by"),

  createdAt: timestamp("created_at").defaultNow(),
});

```

TAREA 2 — Endpoints (`server/routes.ts`)

Preferencias del huésped (permanentes)

GET	/api/guests/:id/preferences	→ Listar preferencias del huésped
POST	/api/guests/:id/preferences	→ Crear preferencia
PATCH	/api/guests/:id/preferences/:prefId	→ Editar preferencia
PATCH	/api/guests/:id/preferences/:prefId/toggle	→ Activar/desactivar
DELETE	/api/guests/:id/preferences/:prefId	→ Eliminar preferencia

Notas de estadía (por reserva)

GET	/api/reservations/:id/stay-notes	→ Listar notas de la estadía
POST	/api/reservations/:id/stay-notes	→ Crear nota
PATCH	/api/reservations/:id/stay-notes/:noteId	→ Editar nota
PATCH	/api/reservations/:id/stay-notes/:noteId/resolve	→ Marcar como resuelta
DELETE	/api/reservations/:id/stay-notes/:noteId	→ Eliminar nota

Alertas de hospitalidad

GET	/api/hospitality/alerts?area=housekeeping	→ Alertas activas por área
GET	/api/hospitality/alerts/reservation/:id	→ Alertas de una reserva específica
PATCH	/api/hospitality/alerts/:id/acknowledge	→ Confirmar que se tomó nota

Vista general del módulo

GET /api/hospitality/dashboard

→ Retorna:

- Huéspedes con check-in hoy que tienen preferencias registradas
- Alertas pendientes de confirmar por área
- Fechas especiales próximas (próximos 7 días)
- Preferencias críticas activas en el hotel (huéspedes con estadía activa)

Generación automática de alertas al check-in

Modificar el endpoint `POST /api/reservations/:id/check-in`:

Al hacer check-in, consultar `guestPreferences` del huésped donde `isActive = true`.

Por cada preferencia activa:

1. Crear un registro en `hospitalityAlerts`.
2. Crear una notificación en `systemNotifications` (del sistema de notificaciones ya especificado) con:

- `type: "hospitality_alert"`
- `targetArea` : según `visibleTo` de la preferencia
- `priority` : según `priority` de la preferencia
- `title` : "Preferencia de huésped — [nombre del huésped]"
- `message` : título y descripción de la preferencia

Las preferencias con `priority = "critical"` generan notificaciones con prioridad `urgent` visible para todas las áreas, independientemente del campo `visibleTo`.

TAREA 3 — Módulo Hospitalidad en el sidebar

(`client/src/pages/hospitality.tsx`)

Nueva página principal del módulo, accesible desde el sidebar bajo "Hospitalidad".

Estructura de la página (tabs)

Tab 1 — Dashboard

Vista de lo que está pasando hoy y esta semana:

- **Huéspedes en casa con preferencias:** Lista de reservas activas (`checked_in`) cuyos huéspedes tienen preferencias registradas. Tarjeta por huésped con nombre, habitación, preferencias destacadas (las de prioridad alta/crítica primero).
- **Alertas pendientes de confirmar:** Lista de `hospitalityAlerts` con `isAcknowledged = false`. Botón "Tomar nota" por cada una.
- **Fechas especiales próximas (7 días):** Huéspedes con check-in en los próximos 7 días que tienen preferencias de categoría `fecha_especial`. Permite preparar detalles con anticipación.
- **Preferencias críticas activas:** Badge destacado si hay algún huésped en casa con preferencia `priority = "critical"` (alergias severas, etc.).

Tab 2 — Preferencias por huésped

Buscador de huéspedes con sus preferencias:

- Input de búsqueda por nombre o código de huésped.
- Al seleccionar un huésped, muestra su ficha completa de preferencias organizadas

por categoría.

- Botón “Agregar preferencia” que abre el formulario de nueva preferencia.
- Cada preferencia muestra: categoría, título, descripción, prioridad, visible para, fecha de registro, quién la registró.
- Indicador de si la preferencia viene de una estadía anterior (con link a esa reserva).
- Toggle para activar/desactivar preferencias que ya no aplican.

Tab 3 — Notas de estadía activas

Lista de notas de las reservas con check-in activo (`checked_in`):

- Filtro por área (`visibleTo`).
- Filtro por estado: pendientes / resueltas.
- Cada nota muestra: huésped, habitación, categoría, descripción, prioridad, área.
- Botón “Marcar como resuelta” para notas de acción.

Tab 4 — Historial

Búsqueda de preferencias y notas históricas por huésped, fecha o categoría. Solo lectura.

TAREA 4 — Integración en otros módulos

En el perfil del huésped (`client/src/pages/guests.tsx`)

Agregar tab “Hospitalidad” en el detalle del huésped:

- Lista de todas sus preferencias activas e inactivas.
- Botón “Agregar preferencia” directamente desde el perfil.
- Historial de notas de estadías anteriores (solo lectura).

En el check-in (`client/src/pages/check-in.tsx`)

Al seleccionar una reserva para hacer check-in, si el huésped tiene preferencias activas:

- Mostrar un panel de alerta antes de confirmar el check-in: “Este huésped tiene [N] preferencias registradas.”
- Lista de preferencias con íconos por categoría y prioridad.

- Las preferencias críticas se destacan en rojo con ícono de advertencia .
- El recepcionista debe hacer clic en “Entendido — Continuar check-in” para proceder.
- Este clic registra el `acknowledgedBy` en las alertas generadas.

En el restaurante (`client/src/pages/restaurant.tsx`)

Al abrir una mesa o una orden de room service vinculada a una habitación ocupada:

- Si el huésped tiene preferencias de categoría `alimentacion` con `visibleTo` incluyendo `restaurant` :
- Mostrar un badge o tooltip “  Preferencias alimentarias” en la mesa.
- Al hacer clic, mostrar las preferencias relevantes (alergias, dietas, bebida favorita).

En housekeeping (`client/src/pages/housekeeping.tsx`)

Al asignar una tarea de limpieza para una habitación ocupada:

- Si el huésped tiene preferencias de categoría `habitacion` o `servicio` con `visibleTo` incluyendo `housekeeping` :
- Mostrar las preferencias relevantes en el detalle de la tarea (horario preferido de limpieza, almohadas, temperatura, etc.).

En el SPA (`client/src/pages/spa.tsx`)

Al crear un turno para un huésped:

- Si el huésped tiene preferencias de categoría `alimentacion` (alergias a productos) o `amenities` con `visibleTo` incluyendo `spa` :
- Mostrar las preferencias en el detalle del turno como recordatorio para el terapeuta.

TAREA 5 — Formulario de preferencias (componente reutilizable)

Crear un componente `PreferenceForm` reutilizable que se usa en el módulo de Hospitalidad, en el perfil del huésped y en el check-in.

Campos del formulario

- **Categoría** (dropdown obligatorio):

-  Habitación
 -  Alimentación
 -  Amenities
 -  Servicio
 -  Fecha especial
 -  Motivo de viaje
 -  Nota interna
 -  Otro
 - **Subcategoría** (dropdown dinámico según categoría, opcional):
 - Se carga según la categoría seleccionada con opciones predefinidas más opción "Otro".
 - **Título** (texto corto, obligatorio): resumen en una línea.
 - **Descripción** (texto largo, opcional): detalle adicional.
 - **Prioridad** (dropdown):
 - Normal (default)
 - Alta
 - Crítica  (con aviso: "Las preferencias críticas son visibles para todas las áreas")
 - **Visible para** (multi-select):
 - Todas las áreas (default)
 - Recepción / Housekeeping / Restaurante / SPA
 - **Registrado por** (texto libre, nombre del staff).
 - **Estadía de origen** (opcional): link a la reserva durante la cual se detectó.
-

Integración con el chatbot

Cuando el chatbot detecte información sobre preferencias del huésped durante una conversación (por ejemplo: "no como gluten" o "me gusta el piso alto"), puede enviar un

webhook al endpoint:

```
POST /api/guests/:id/preferences
```

Con los campos correspondientes y `recordedBy: "Chatbot"` . Esto permite que el conocimiento del huésped se acumule automáticamente en su perfil desde múltiples canales.

Notas para Replit

- Implementar en orden: Tarea 1 (schema) → Tarea 2 (endpoints) → Tarea 3 (módulo principal) → Tarea 4 (integraciones) → Tarea 5 (componente reutilizable).
 - La Tarea 4 depende de que el sistema de notificaciones (especificado en el documento Chatbot + Web Check-in) esté implementado, ya que las alertas de check-in usan `systemNotifications` .
 - El campo `visibleTo` es un array de texto en PostgreSQL. En MemStorage implementarlo como `string[]` .
 - Los íconos de categoría son solo sugerencias visuales para el frontend — no son parte del schema.
 - La integración con restaurante (mostrar preferencias alimentarias en la mesa) es especialmente importante desde el punto de vista de seguridad del huésped (alergias). Darle prioridad dentro de la Tarea 4.
 - Mantener la nomenclatura existente (camelCase en TypeScript, snake_case en SQL).
 - Los textos de interfaz deben estar en español, consistente con el resto del sistema.
-

*Documento generado para el proyecto Maran Suite System — Módulo Hospitalidad
Este módulo completa la etapa operativa del sistema.*